

4.1 Thiết kế kiến trúc

4.1.1 Lựa chọn kiến trúc phần mềm

Hệ thống được xây dựng theo **kiến trúc ba tầng kinh điển (three-tier)** — gồm tầng trình diễn, tầng nghiệp vụ và tầng dữ liệu — trên nền mô hình Client–Server, kết hợp với **kiến trúc phân lớp (Layered Architecture)** ở phía máy chủ. Cụ thể, phía máy chủ áp dụng một biến thể mở rộng của mẫu **MVC** bằng cách bổ sung thêm *tầng dịch vụ* (Service) — thường gọi là **MVC + Service (MVCS)** — nhằm tách biệt rõ ràng logic nghiệp vụ khỏi tầng điều khiển.

Ý tưởng cốt lõi của kiến trúc phân lớp là chia hệ thống thành các tầng có trách nhiệm tách biệt, mỗi tầng chỉ phụ thuộc vào tầng ngay bên dưới nó. Điều này giúp giảm sự ràng buộc (loose coupling), tăng khả năng tái sử dụng, kiểm thử và bảo trì. Mẫu MVC truyền thống gồm ba thành phần: *Model* (dữ liệu và quy tắc trên dữ liệu), *View* (trình bày) và *Controller* (điều phối tương tác).

Áp dụng cụ thể vào hệ thống, các thành phần lý thuyết được ánh xạ thành các thành phần thực tế như sau:

- **Model (M):** gồm 12 lược đồ Mongoose trong gói `models` — ví dụ `User`, `FoodDonation`, `FoodRequest`, `Delivery`, `Conversation`, `Message`, `Feedback`, `Notification`, `OtpSession` và ba lược đồ hồ sơ `DonorProfile`, `ReceiverProfile`, `VolunteerProfile`. Đây là nơi định nghĩa cấu trúc dữ liệu và ràng buộc.
- **View (V):** do phía client đảm nhiệm — ứng dụng di động *React Native* và trang quản trị *React* — vì máy chủ là dịch vụ REST không trạng thái, chỉ trả về dữ liệu JSON để client kết xuất giao diện.
- **Controller (C):** gồm các lớp trong gói `controllers` (`authController`, `foodDonationController`, `foodRequestController`, ...), chịu trách nhiệm nhận yêu cầu HTTP, kiểm tra đầu vào sơ bộ, gọi tầng dịch vụ và định dạng phản hồi.
- **Service (S) — tầng bổ sung:** gồm các lớp trong gói `services` chứa toàn bộ logic nghiệp vụ (ví dụ `AuthService`, `FoodDonationService` cùng các mô-đun con `donorActions`, `receiverActions`, `volunteerActions`, ...). Việc tách tầng này giúp Controller mỏng, dễ thay đổi nghiệp vụ mà không ảnh hưởng định tuyến.

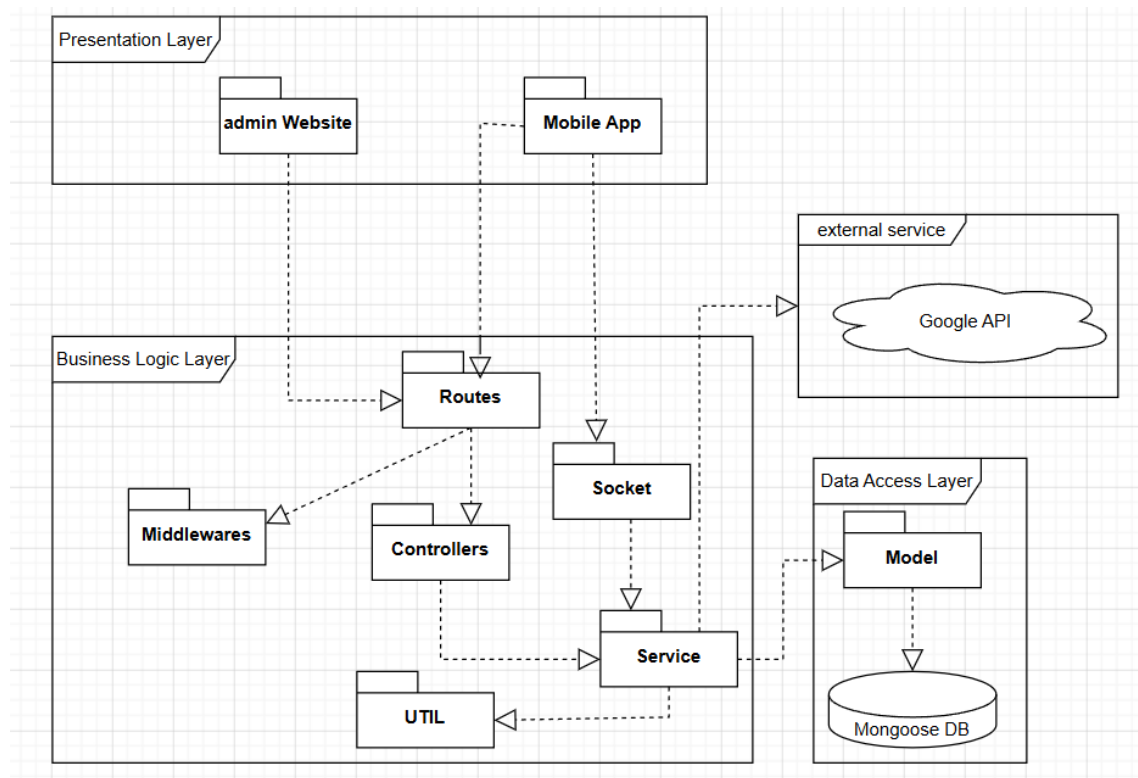
Bên cạnh đó, hệ thống bổ sung các thành phần xuyên suốt (cross-cutting)

so với MVC thuần: gói **middlewares** (xác thực JWT, phân quyền admin, xử lý tải tệp), gói **socket** (kênh thời gian thực Socket.IO), gói **utils**, **i18n** và **config**.

Ở phía **ứng dụng di động**, do đặc thù React, kiến trúc được tổ chức theo hướng **thành phần (component-based)** kết hợp **luồng dữ liệu một chiều** kiểu Flux thông qua thư viện quản lý trạng thái *Zustand*. Các màn hình (**app/**) đóng vai trò View, tầng **src/api** đóng vai trò gọi dịch vụ, tầng **src/store** giữ trạng thái toàn cục, còn **src/types** định nghĩa kiểu dữ liệu dùng chung.

4.1.2 Thiết kế tổng quan

Hình 4.1 trình bày biểu đồ gói tổng quan theo **kiến trúc ba tầng kinh điển (three-tier)**. Các gói được phân vào ba tầng — trình diễn, nghiệp vụ và dữ liệu — cùng một nhóm dịch vụ ngoài; mọi quan hệ phụ thuộc (mũi tên nét đứt) đều hướng từ tầng trên xuống tầng dưới.



Hình 4.1: Biểu đồ gói tổng quan của hệ thống

Nhiệm vụ của từng tầng và các gói thành phần:

- **Tầng trình diễn (Presentation):** *Mobile App* (React Native/Expo) phục vụ ba vai trò người dùng cuối và *Admin Web* (React/Vite) phục vụ quản trị viên. Mobile gọi REST qua **routes** và mở kênh WebSocket qua

socket; Admin Web gọi REST.

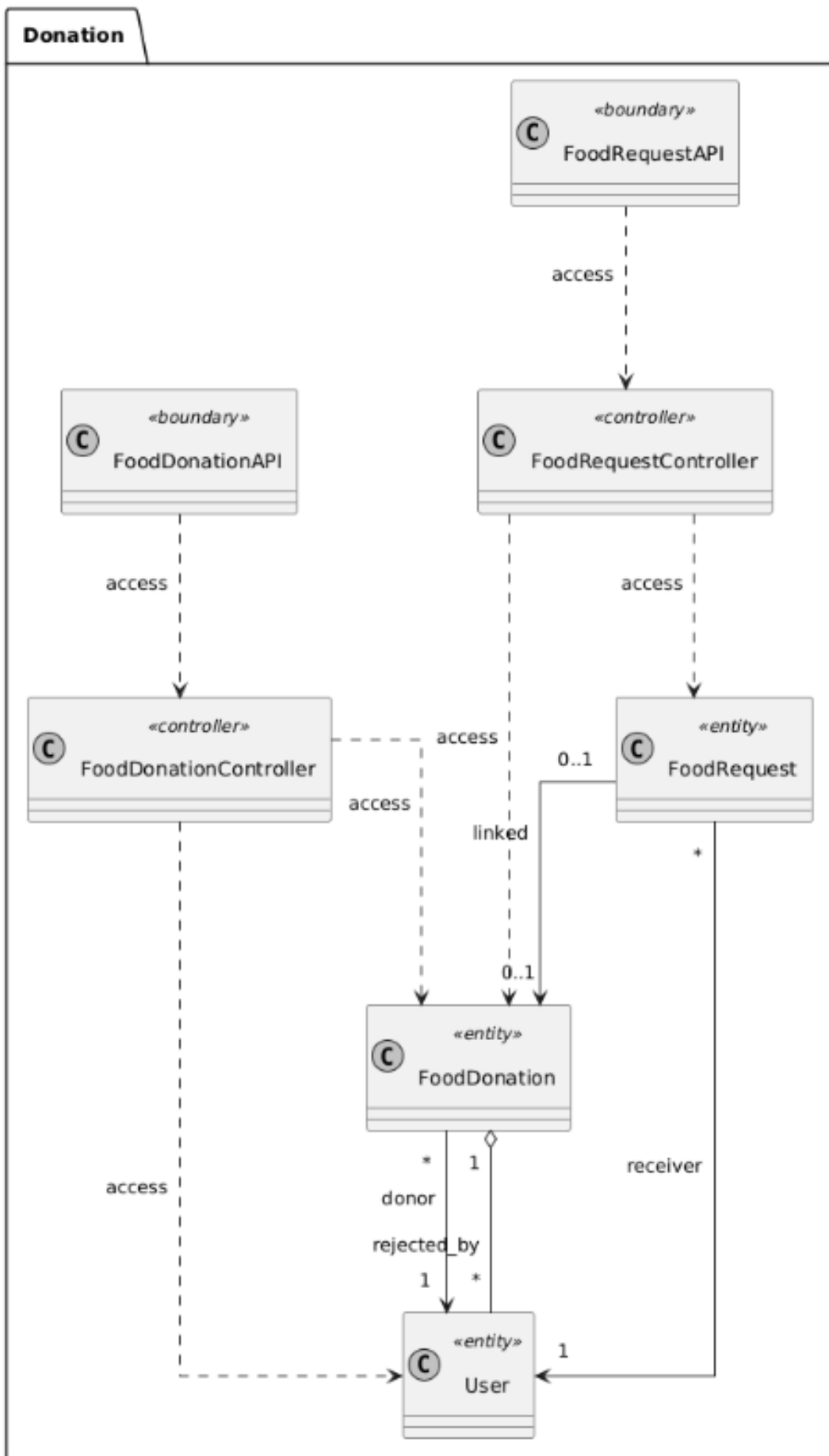
- **Tầng nghiệp vụ (Business Logic):** là toàn bộ máy chủ phụ trợ, gồm `routes` (khai báo điểm cuối), `controllers` (điều phối request/response), `socket` (điểm vào thời gian thực), `middlewares` (xác thực JWT, phân quyền, tải tệp), `services` chứa logic nghiệp vụ theo từng nhóm chức năng (xác thực, người dùng/hồ sơ, quyên góp & giao nhận, yêu cầu thực phẩm, trò chuyện, thông báo, đánh giá) và `utils/config` (tiện ích dùng chung).
- **Tầng dữ liệu (Data):** gồm `models` (lược đồ Mongoose — đóng vai trò truy cập dữ liệu) và cơ sở dữ liệu MongoDB.
- **Dịch vụ ngoài:** *Expo Push* (thông báo đẩy) và *Google Distance Matrix* (khoảng cách đường bộ), được tầng nghiệp vụ gọi tới.

Thiết kế tuân thủ các quy tắc của kiến trúc phân tầng: (i) **không có phụ thuộc vòng/lăn nhau** giữa các gói; (ii) **tầng dưới không phụ thuộc tầng trên** — mọi mũi tên đều hướng xuống (trình diễn → nghiệp vụ → dữ liệu); (iii) **không phụ thuộc bỏ qua tầng** — tầng trình diễn không truy cập thẳng tầng dữ liệu mà phải thông qua tầng nghiệp vụ. Các dịch vụ ngoài là hệ thống bên thứ ba, được tầng nghiệp vụ tích hợp qua mạng nên được biểu diễn tách biệt khỏi ba tầng chính.

4.1.3 Thiết kế chi tiết gói

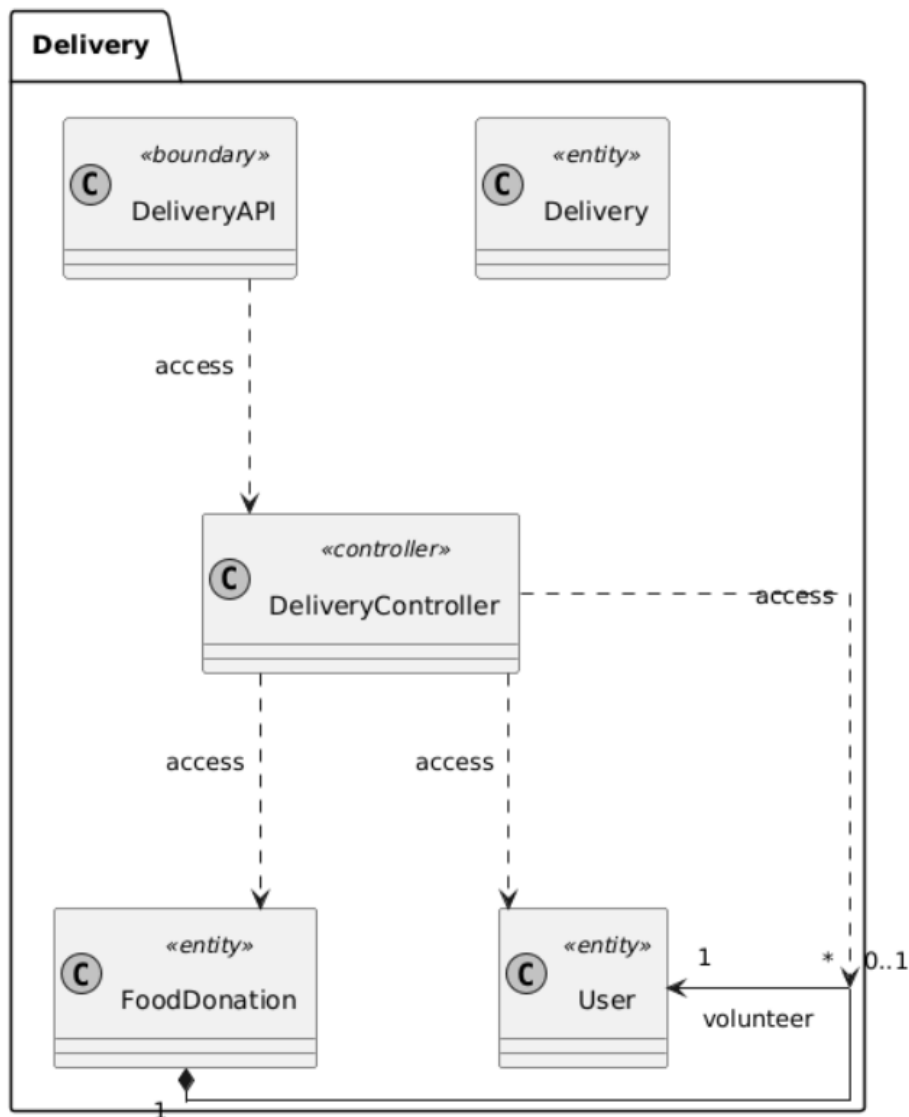
Vì hệ thống gồm nhiều gói, phần thiết kế chi tiết được tổ chức theo **nhóm gói liên quan** (lát cắt chức năng) thay vì mỗi gói một biểu đồ, gồm ba nhóm: (1) Quyên góp & giao nhận, (2) Tài khoản & xác thực, (3) Trò chuyện & thông báo. Mỗi hình chỉ ghi tên lớp (sắp theo từng gói) và tập trung thể hiện quan hệ giữa các lớp (phụ thuộc, kết hợp, kết tập, hợp thành).

Do phạm vi rộng, nhóm **quyên góp & giao nhận** được tách thành hai biểu đồ: Hình 4.2 mô tả phần **đăng & ghép đơn**, Hình 4.3 mô tả phần **giao nhận**. Cả hai cùng dùng lớp mặt tiền (facade) `FoodDonationService` ủy quyền cho các mô-đun chuyên trách, giúp tách một tệp nghiệp vụ rất dài thành nhiều mô-đun nhỏ, mỗi mô-đun một trách nhiệm.



Hình 4.2: Thiết kế chi tiết: Quyên góp (đăng & ghép đơn)

Ở phần đăng & ghép (Hình 4.2): FoodDonationController/FoodRequestController ủy quyền cho facade; donorActions, receiverActions và queries thao tác trên FoodDonation; FoodRequestService quản lý FoodRequest và dùng locationFilter/distance để gợi ý theo vị trí. Các mũi tên nét đứt (controller – service – mô-đun – model) thể hiện quan hệ *phụ thuộc*. Về quan hệ thực thể: *kết hợp* (FoodDonation–User theo donor, FoodRequest–User theo receiver, FoodRequest–FoodDonation khi đơn tạo từ yêu cầu) và *kết tập* (FoodDonation chứa danh sách rejected_by gồm các User tồn tại độc lập).

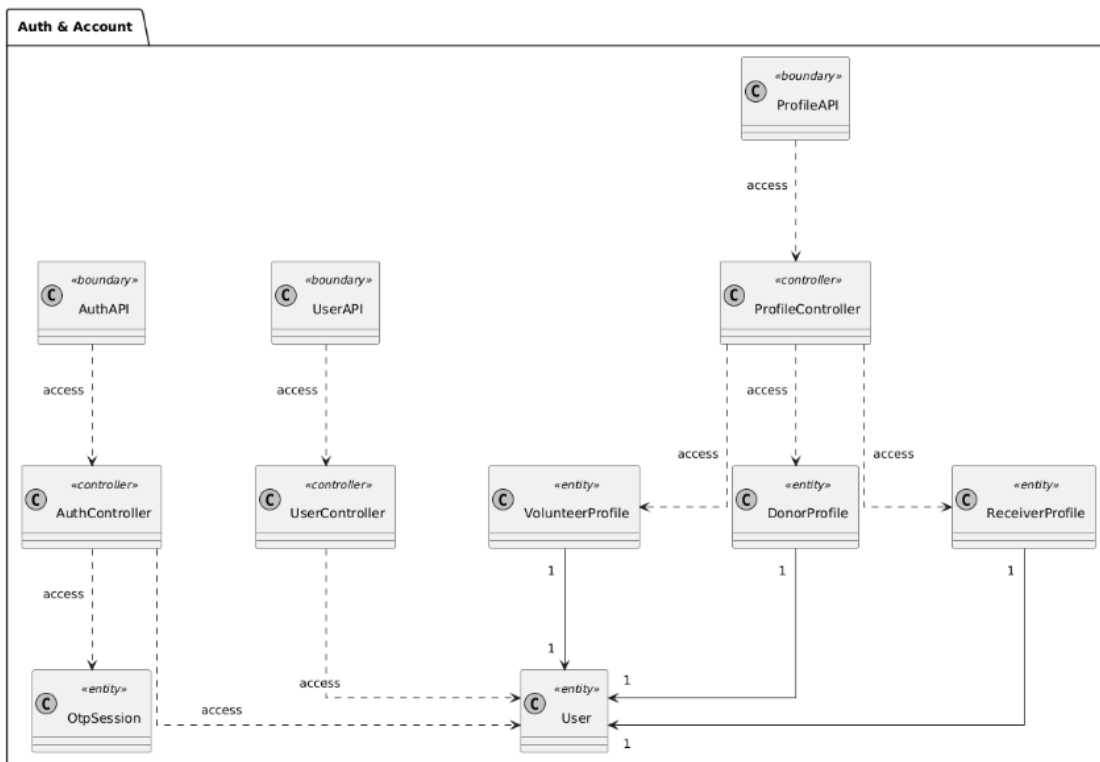


Hình 4.3: Thiết kế chi tiết: Giao nhận (delivery)

Ở phần giao nhận (Hình 4.3): facade ủy quyền cho volunteerActions (nhận/giao đơn), receiverConfirm (xác nhận đã nhận), maintenance (tác vụ tự động: hết hạn, tự xác nhận, no-show) và receiverActions (tự lấy

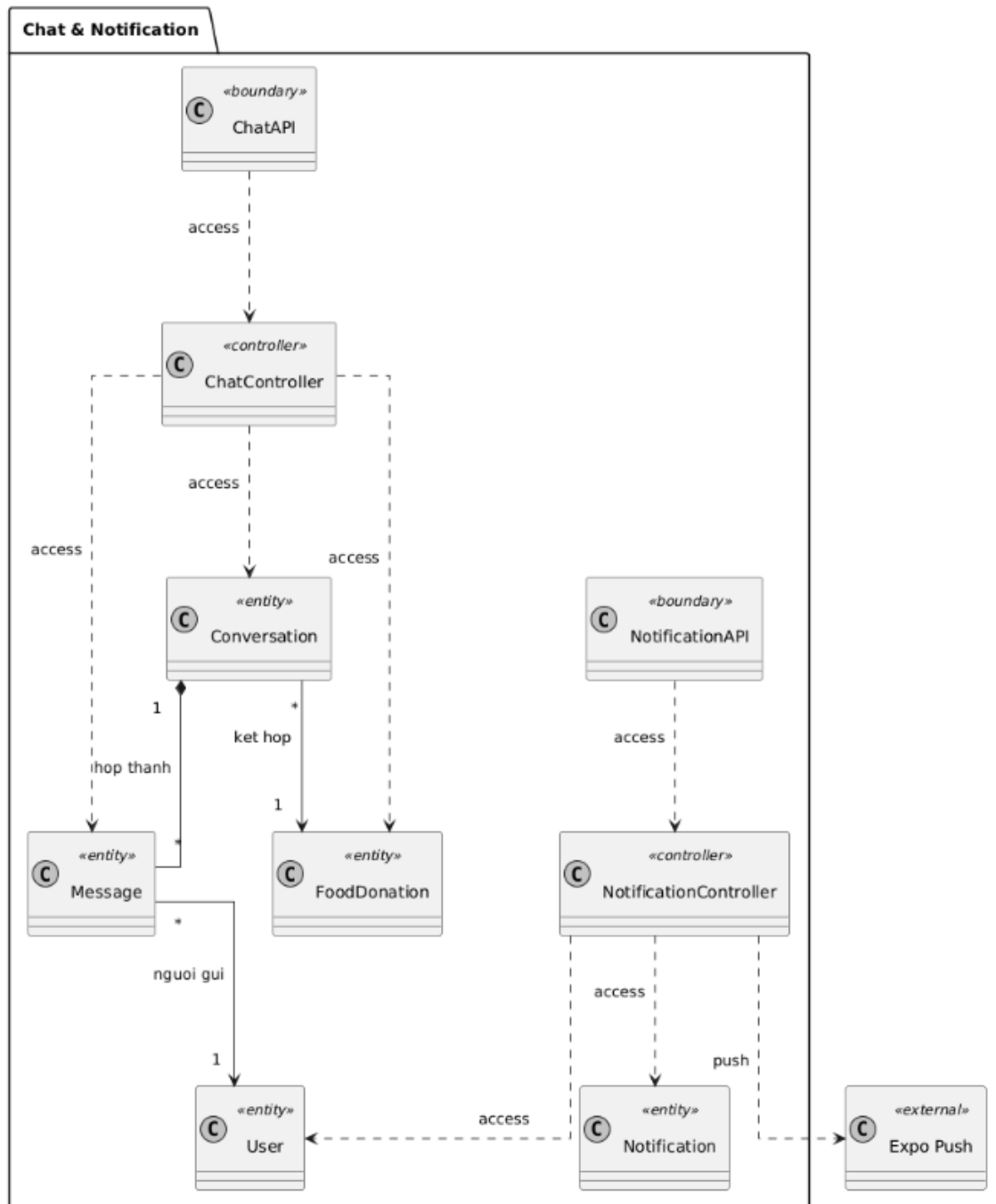
hàng). Các mô-đun thao tác trên **Delivery**, dùng **pickupCode** để xác thực mã lấy hàng và **points** để cộng điểm cho **User**. Quan hệ *hợp thành* (hình thoi đặc): mỗi **FoodDonation** sở hữu tối đa một **Delivery** (bản ghi giao nhận chỉ phát sinh khi người nhận chọn phương thức nhận) và gắn liền vòng đời với đơn — đơn bị huỷ thì delivery cũng bị huỷ. Miền nghiệp vụ này không dùng quan hệ *kế thừa* hay *thực thi* do thiết kế theo mô-đun hàm và lược đồ tài liệu (document); quan hệ tổng quát hoá giữa các vai trò người dùng đã được trình bày ở mô hình use case (Chương 2).

Hình 4.4 đặc tả nhóm gói **tài khoản & xác thực**. Mỗi bộ định tuyến gắn **authMiddleware** rồi gọi controller tương ứng; controller ủy quyền cho service. **AuthService** dùng **otpUtil** và thao tác trên **User**, **OtpSession**; **ProfileService** quản lý ba loại hồ sơ. Ba lược đồ hồ sơ có quan hệ *kết hợp* một-một với **User** (qua **user_id**).



Hình 4.4: Thiết kế chi tiết nhóm gói “Tài khoản & xác thực”

Hình 4.5 đặc tả nhóm gói **trò chuyện & thông báo**. Tin nhắn được gửi qua hai đường: REST (ChatController) và thời gian thực (chatSocket), cùng đi tới ChatService; khi có tin mới, ChatService gọi NotificationService để đẩy thông báo qua *Expo Push*. Về quan hệ thực thể, một **Conversation** *hợp thành* nhiều **Message** (Message không tồn tại độc lập) và *kết hợp* với một **FoodDonation**.



Hình 4.5: Thiết kế chi tiết nhóm gói “Trò chuyện & thông báo”

Hai gói *xuyên suốt* `middlewares` và `utils/i18n/config` xuất hiện trong nhiều nhóm trên với vai trò hỗ trợ (xác thực, tải tệp, đa ngôn ngữ, cấu hình) nên không vẽ thành biểu đồ chi tiết riêng.

4.2 Thiết kế chi tiết

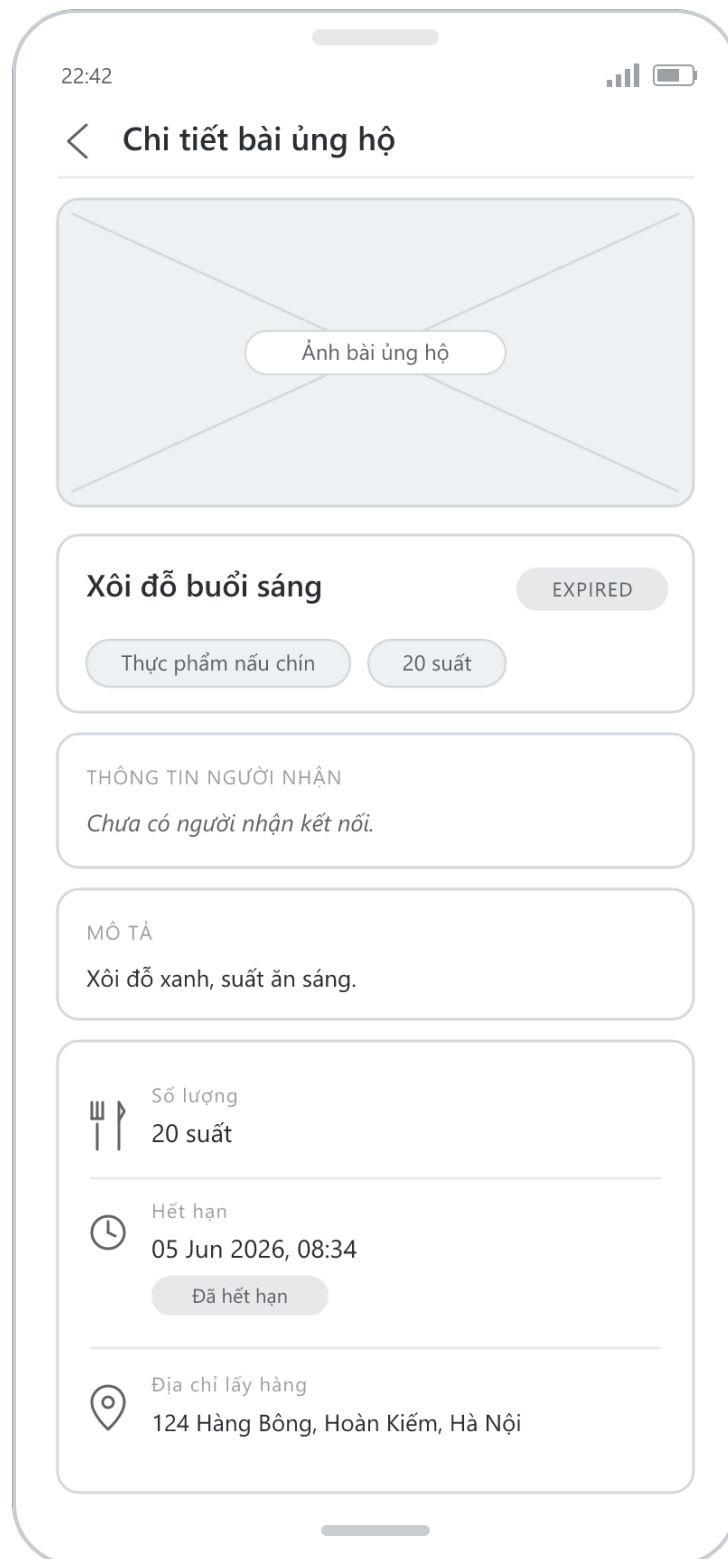
4.2.1 Thiết kế giao diện

Màn hình mục tiêu. Ứng dụng hướng tới điện thoại thông minh chạy **Android**, ở chế độ dọc (portrait). Kích thước màn hình phổ biến từ 5.5" đến 6.7", độ phân giải từ HD 720×1280 đến FHD+ 1080×2400, mật độ điểm

ảnh trải từ *hdpi* đến *xxxhdpi*. Nhờ React Native sử dụng đơn vị *dp* (density-independent pixels), giao diện tự co giãn theo mật độ điểm ảnh nên hiển thị nhất quán trên nhiều thiết bị. Bảng màu sử dụng độ sâu màu 24-bit (hơn 16,7 triệu màu).

Các thống nhất khi thiết kế.

- **Phối màu theo vai trò:** mỗi vai trò (Donor/Receiver/Volunteer) có một màu nhấn riêng được định nghĩa tập trung trong `theme/roleUi.ts`, giúp người dùng nhận biết ngữ cảnh đang thao tác.
- **Điều hướng:** sử dụng thanh tab dưới (bottom tabs) cho các mục chính (Trang chủ, Tin nhắn, Thông báo, Cá nhân) và ngăn xếp (stack) cho các luồng chi tiết, được tổ chức theo cơ chế định tuyến theo thư mục của `expo-router`.
- **Nút và điều khiển:** nút bo góc; nút hành động chính tô màu nhấn của vai trò, nút phụ dùng viền; các điều khiển nhập liệu có nhãn và trạng thái lỗi rõ ràng.
- **Vị trí thông điệp phản hồi:** thông báo ngắn dùng *toast* ở phía trên/dưới màn hình; lỗi nhập liệu hiển thị *inline* ngay dưới trường tương ứng (kèm tô đỏ viền); các xác nhận quan trọng (ví dụ nhập mã lấy hàng) dùng hộp thoại *modal*.
- **Đa ngôn ngữ:** toàn bộ chuỗi hiển thị lấy từ mô-đun `i18n`, hỗ trợ Tiếng Việt và Tiếng Anh.



Hình 4.6: Khung wireframe chi tiết bài ủng hộ

22:45

Loại bài đăng: Ủng hộ

Thêm tiêu đề

Thêm tiêu đề thực phẩm

Mô tả:

Ví dụ: cà chua từ vườn.
Cung cấp càng nhiều chi tiết càng tốt để tăng

Loại thực phẩm

Thực phẩm nấu chín

Số lượng thực phẩm

suất

-

5

+

Hình ảnh:



+Thêm nhiều hơn

Ngày hết hạn

DD MMM YYYY



Thời gian hết hạn

HH:MM AM/PM



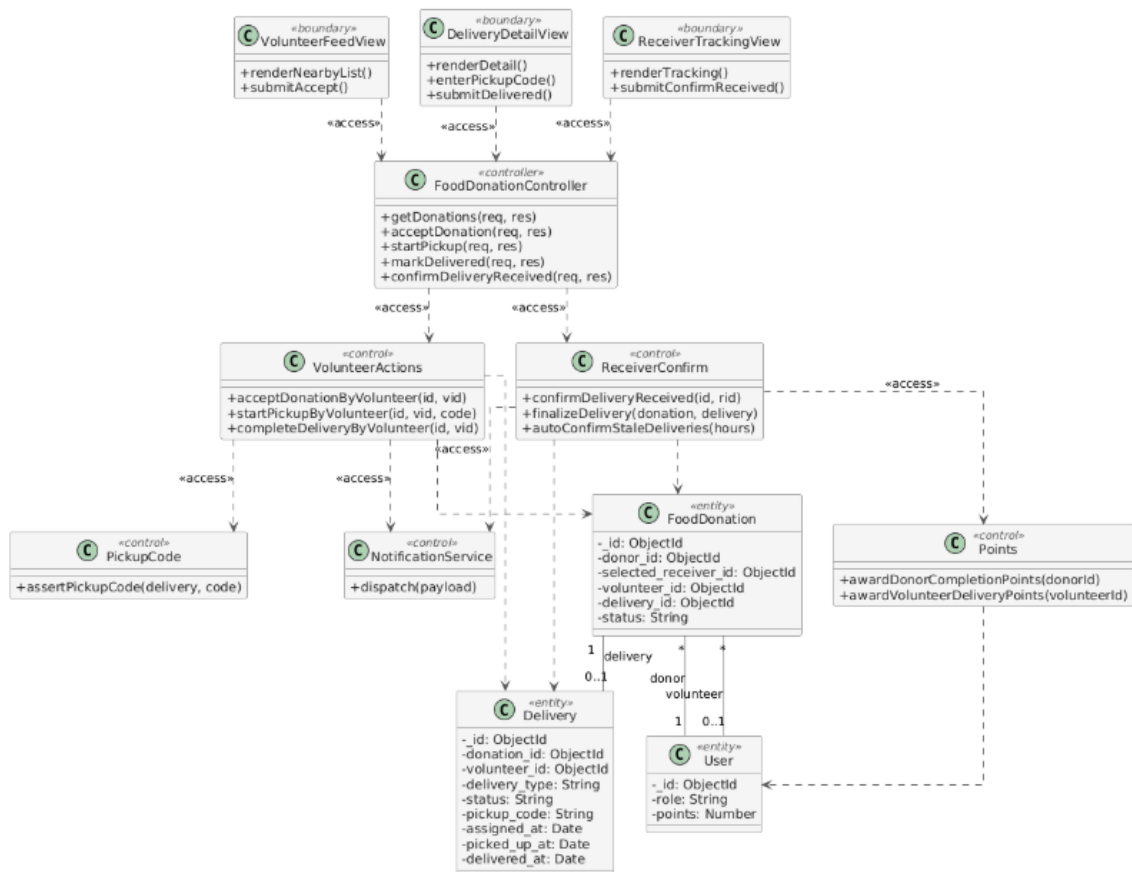
☐ Tôi đảm bảo rằng chất lượng thực phẩm và vệ sinh đã được duy trì

Gửi

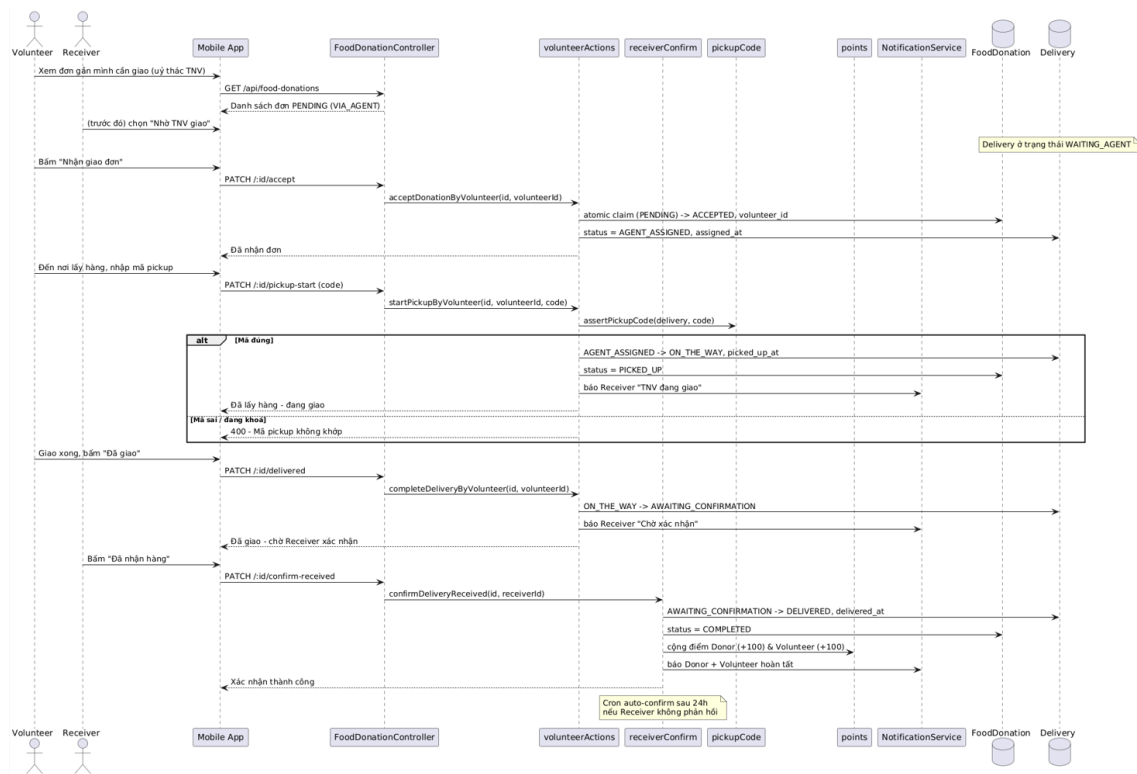
Hình 4.7: Khung wireframe tạo bài ủng hộ thực phẩm

4.2.2 Thiết kế lớp

a. Thiết kế chi tiết lớp: Nhận và giao đơn thông qua tình nguyện viên

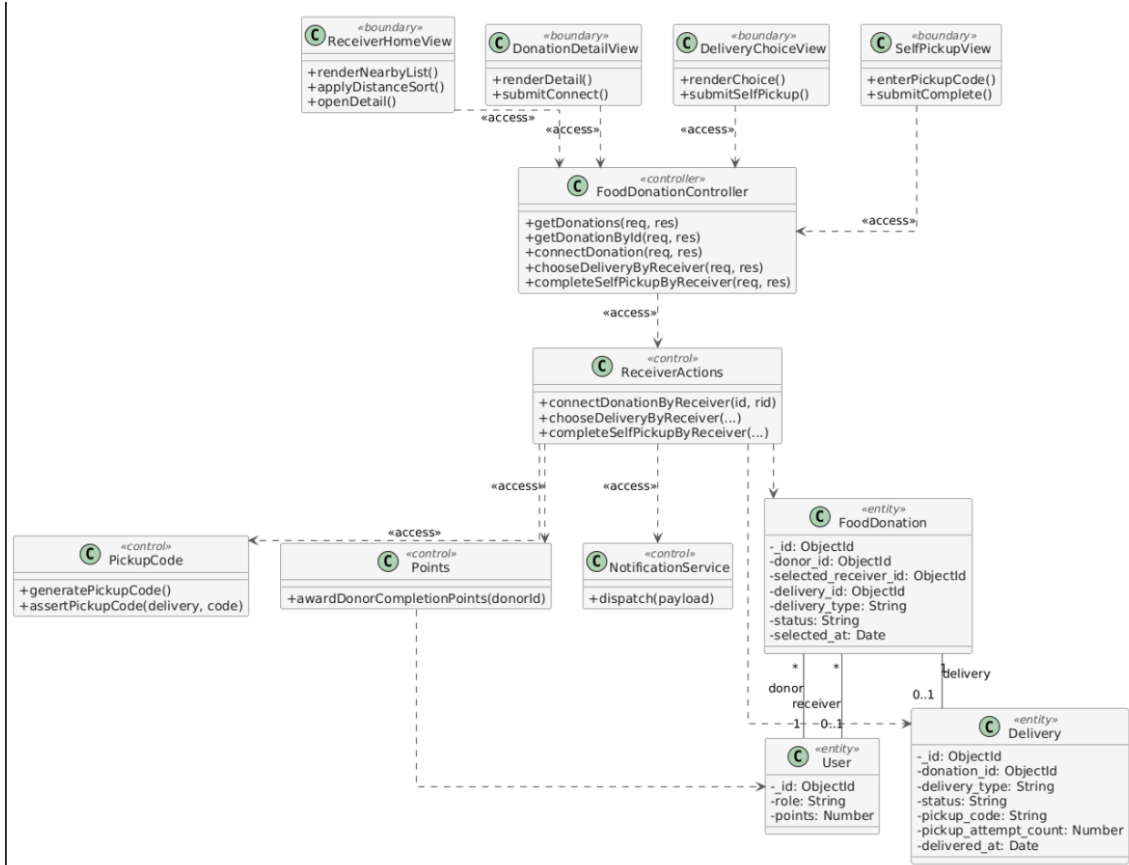


Hình 4.8: Biểu đồ lớp chi tiết “Nhận và giao đơn thông qua tình nguyện viên”

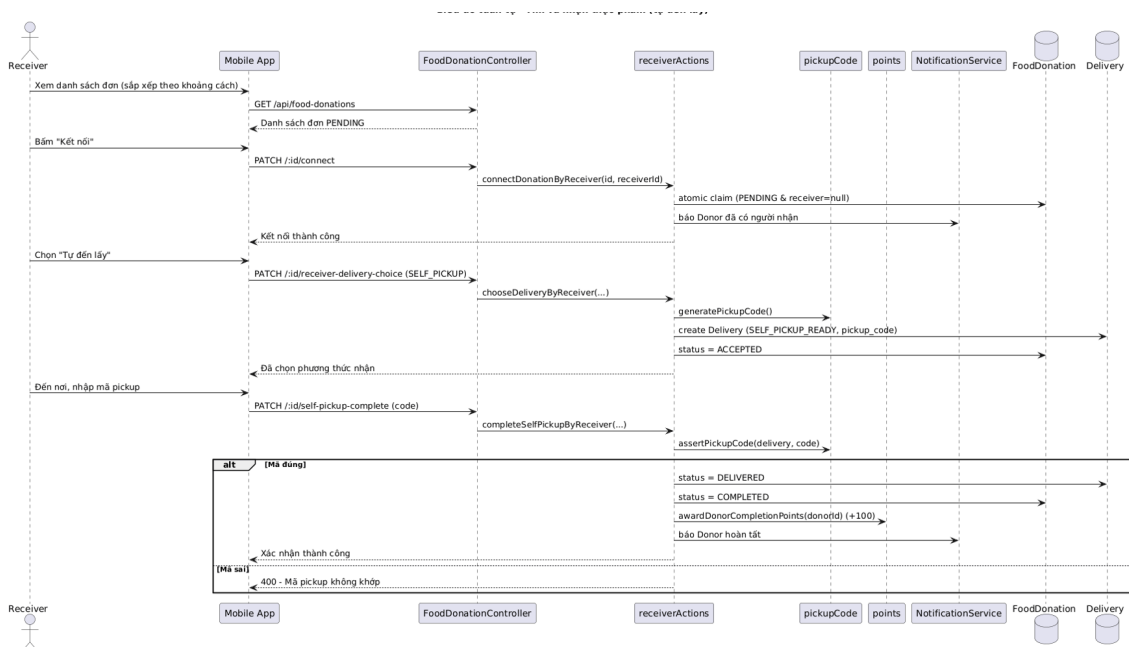


Hình 4.9: Biểu đồ tuần tự “Nhận và giao đơn qua tình nguyện viên”

b. Thiết kế chi tiết lớp: Tìm và tự đến nhận thực phẩm của người nhận (Receiver)

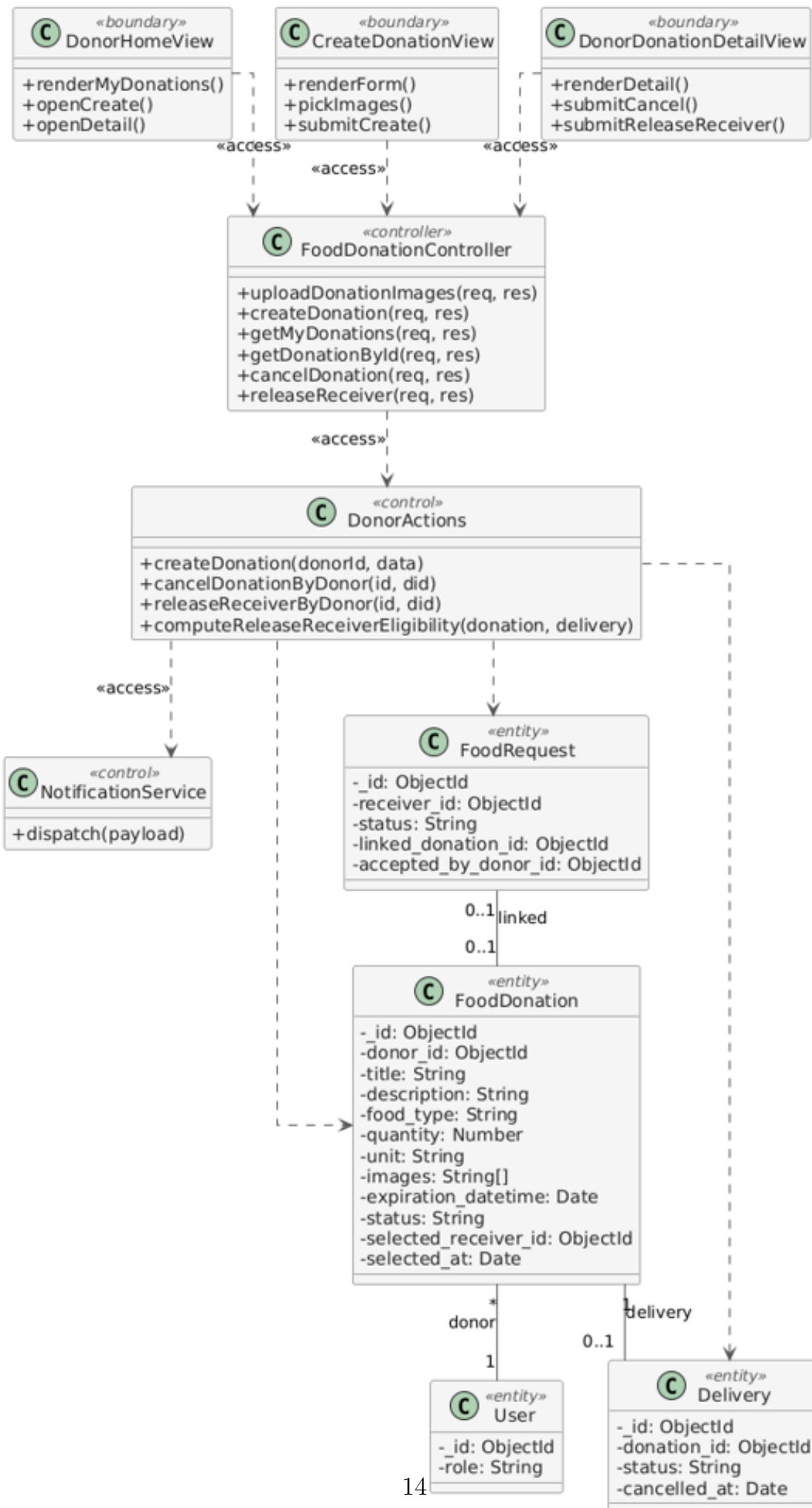


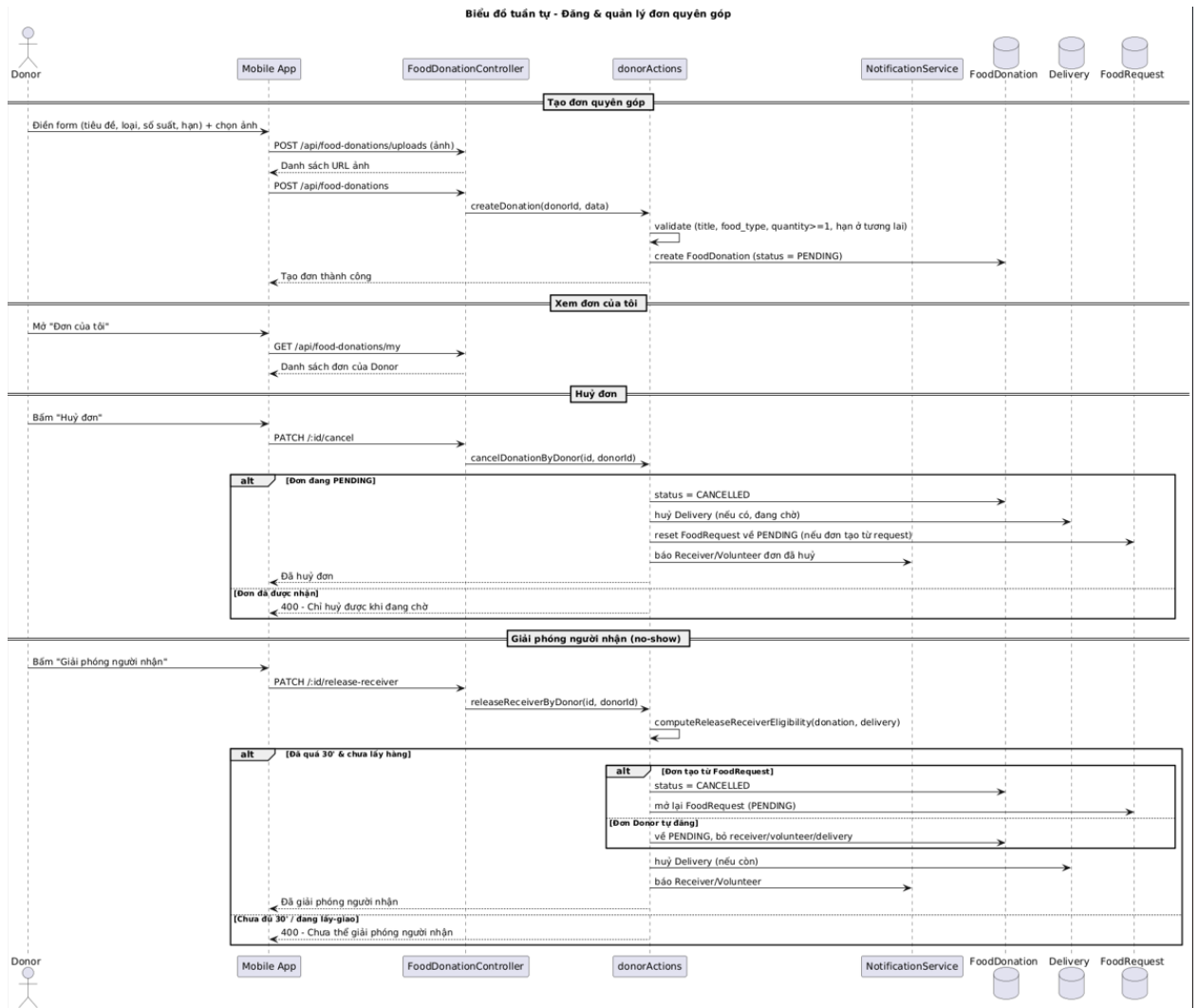
Hình 4.10: Biểu đồ lớp chi tiết “Nhận và giao đơn thông qua tình nguyện viên”



Hình 4.11: Biểu đồ tuần tự “Nhận và giao đơn qua tình nguyện viên”

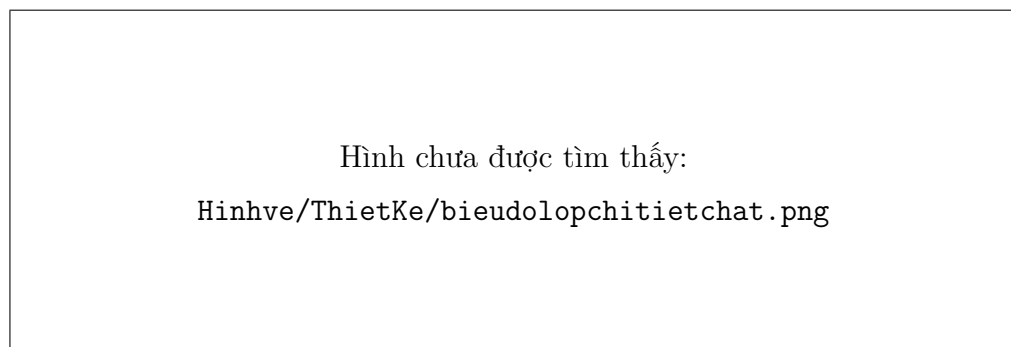
c. Thiết kế chi tiết lớp: Đăng và quản lý đơn quyên góp của người cho (Donor)





Hình 4.13: Biểu đồ tuần tự “Nhận và giao đơn qua tình nguyện viên”

d. Thiết kế chi tiết lớp: Trao đổi qua tin nhắn



Hình 4.14: Biểu đồ lớp chi tiết “Trao đổi qua tin nhắn”

Các liên kết chính: mỗi **User** có quan hệ một-một (tùy chọn) với một trong ba hồ sơ **DonorProfile/ReceiverProfile/VolunteerProfile** tùy vai trò. Một **User** (vai trò **Donor**) tạo nhiều **FoodDonation**; một **User** (vai trò **Receiver**) tạo nhiều **FoodRequest**. Mỗi **FoodDonation** có thể sinh ra một **Delivery**; một **FoodRequest** khi được đáp ứng sẽ liên kết tới một **FoodDonation** (**linked_donation_id**). **Conversation** gắn với một **FoodDonation** và chứa nhiều **Message**. **Feedback** gắn với một **Delivery** và hai người dùng (người đánh giá, người được đánh giá). **Notification** thuộc về một **User**. **OtpSession** là tập độc lập (định danh theo số điện thoại).

Một số ràng buộc/chỉ mục đáng chú ý ở tầng cơ sở dữ liệu:

- **User**: chỉ mục duy nhất (unique, sparse) trên **email** và **phone_number**; ba hồ sơ có chỉ mục duy nhất trên **user_id** để bảo đảm quan hệ 1-1.
- **OtpSession**: chỉ mục TTL trên **expires_at** để tự xóa phiên hết hạn.
- **Conversation**: chỉ mục duy nhất (**donation_id**, **pair_key**) để mỗi cặp người dùng chỉ có một hội thoại trên một đơn; chỉ mục (**participants**, **last_message_at**) để truy vấn danh sách hội thoại nhanh.
- **Message**: chỉ mục (**conversation_id**, **createdAt**) phục vụ phân trang và (**conversation_id**, **sender_id**, **seen_by**) để đếm tin chưa đọc.

4.3 Xây dựng ứng dụng

4.3.1 Thư viện và công cụ sử dụng

Bảng 4.1 liệt kê các công nghệ, thư viện và công cụ chính kèm phiên bản sử dụng.

Thành phần	Công cụ / Thư viện	Phiên bản
Máy chủ (runtime)	Node.js	20.x (LTS)
Web framework	Express	5.2.1
ODM cơ sở dữ liệu	Mongoose	9.1.5
Cơ sở dữ liệu	MongoDB (Atlas)	7.x
Thời gian thực	Socket.IO	4.8.3
Xác thực / Băm	jsonwebtoken / bcryptjs	9.0.3 / 3.0.3
Tải tệp / Ghi log	multer / morgan	2.1.1 / 1.10.1
Ứng dụng di động	React Native / Expo SDK	0.81.5 / 54.0.33
Định tuyến / Trạng thái	expo-router / Zustand	6.0.23 / 5.0.11
Gọi API / Realtime client	Axios / socket.io-client	1.13.5 / 4.8.3
Bản đồ / Định vị	react-native-maps / expo-location	1.20.1 / 19.0.8
Thông báo / Lưu token	expo-notifications / expo-secure-store	0.32.17 / 15.0.8
Ngôn ngữ (mobile/web)	TypeScript	5.9.2 / 5.7.2
Trang quản trị	React / Vite	18.3.1 / 6.0.7
Biểu đồ / Icon (web)	Recharts / lucide-react	2.15.0 / 0.469.0
IDE	Visual Studio Code	(bản mới nhất)
Quản lý mã nguồn	Git / GitHub	—
Kiểm thử API	Postman	(bản mới nhất)
Đóng gói / Triển khai	Docker / Railway	—
Vẽ biểu đồ	PlantUML	—

Bảng 4.1: Danh sách thư viện và công cụ sử dụng

4.3.2 Kết quả đạt được

Hệ thống hoàn chỉnh gồm ba sản phẩm được đóng gói:

- **Máy chủ phụ trợ (Backend):** dịch vụ REST API + WebSocket, được đóng gói thành *Docker image* để triển khai.
- **Ứng dụng di động (Mobile):** gói cài đặt *APK* cho Android, phục vụ ba vai trò người dùng.
- **Trang quản trị (Admin Web):** ứng dụng web tĩnh (Vite build) cho quản trị viên.

Bảng 4.2 thống kê quy mô mã nguồn của từng sản phẩm (không tính thư viện bên thứ ba/node_modules).

Sản phẩm	Số tệp mã nguồn	Số dòng lệnh	Dung lượng
Backend (Node.js)	57	5.639	269,6 KB
Mobile (React Native)	88	15.440	630,2 KB
Admin Web (React)	27	3.528	110,8 KB
Tổng cộng	172	24.607	≈ 1,0 MB

Bảng 4.2: Thống kê quy mô mã nguồn

Ngoài ra, phía máy chủ định nghĩa **12 lược đồ dữ liệu (model)** và được tổ chức thành các nhóm gói: `routes`, `controllers`, `services` (kèm gói `con foodDonation`), `models`, `middlewares`, `utils`, `index`, `config`, `socket`.

4.3.3 Minh họa các chức năng chính

[Chèn ảnh chụp màn hình các chức năng chính kèm chú thích ngắn: (1) Đăng ký/Đăng nhập & OTP; (2) Đăng đơn quyên góp; (3) Danh sách & chi tiết đơn theo khoảng cách; (4) Chọn phương thức nhận & mã lấy hàng; (5) Theo dõi giao hàng theo thời gian thực; (6) Nhắn tin; (7) Điểm thưởng & bảng xếp hạng; (8) Trang quản trị web.]

4.4 Kiểm thử

Do hệ thống chưa tích hợp bộ kiểm thử tự động, công tác kiểm thử được thực hiện chủ yếu theo phương pháp **kiểm thử hộp đen (black-box)** ở mức chức năng, kết hợp **kiểm thử API** bằng Postman và **kiểm thử thủ công/khám phá** trên thiết bị thật và máy ảo Android. Các kỹ thuật thiết kế ca kiểm thử gồm *phân hoạch tương đương* và *phân tích giá trị biên* (ví dụ: số lượng ≥ 1 , mật khẩu ≥ 6 ký tự, OTP tối đa 5 lần thử).

Bảng 4.3 và Bảng 4.4 trình bày một số ca kiểm thử tiêu biểu cho hai chức năng quan trọng nhất.

Mã	Mô tả	Dữ liệu vào	Kết quả mong đợi	KQ
TC01	Đăng ký với dữ liệu hợp lệ	SĐT mới, mật khẩu ≥ 6 ký tự	Tạo tài khoản, gửi OTP	Đạt
TC02	SĐT đã đăng ký & xác minh	SĐT đã tồn tại	Báo lỗi “đã được đăng ký”	Đạt
TC03	Mật khẩu quá ngắn	Mật khẩu 3 ký tự	Báo lỗi, không tạo	Đạt
TC04	Nhập đúng OTP còn hạn	OTP đúng	Xác minh, trả JWT	Đạt
TC05	Nhập sai OTP	OTP sai	Tăng số lần thử, báo còn lại	Đạt
TC06	Nhập OTP quá 5 lần	Sai 5 lần	Khoá, yêu cầu mã mới	Đạt
TC07	OTP hết hạn (>10 phút)	OTP cũ	Báo hết hạn	Đạt

Bảng 4.3: Ca kiểm thử chức năng “Đăng ký & xác minh OTP”

Mã	Mô tả	Dữ liệu vào	Kết quả mong đợi	KQ
TC08	Kết nối đơn PENDING	Receiver bấm kết nối	Chốt người nhận, báo Donor	Đạt
TC09	Hai receiver kết nối đồng thời	2 yêu cầu cùng lúc	Chỉ 1 thắng, người kia 409	Đạt
TC10	Kết nối đơn của chính mình	Donor = Receiver	Bị từ chối	Đạt
TC11	Tự lấy: nhập đúng mã pickup	Mã đúng	Hoàn tất, cộng 100 điểm Donor	Đạt
TC12	Tự lấy: nhập sai mã pickup	Mã sai	Báo lỗi, không hoàn tất	Đạt
TC13	Báo TNV không đến khi đang giao	Trạng thái ON_THE_WAY	Hủy đơn, báo các bên	Đạt

Bảng 4.4: Ca kiểm thử chức năng “Tìm và nhận thực phẩm”

Tổng kết. Đã thực hiện 13 ca kiểm thử tiêu biểu cho hai chức năng trọng yếu (chi tiết các ca khác đặt ở phụ lục), tất cả đều **Đạt**. Các nhánh xử lý ngoại lệ (tranh chấp đồng thời, sai/hết hạn mã, vượt số lần thử) đều cho kết quả đúng như thiết kế nhờ các phép cập nhật nguyên tử và cơ chế chống dò mã ở tầng dịch vụ.

4.5 Triển khai

Hệ thống được triển khai theo mô hình điện toán đám mây với các thành phần tách biệt:

- **Máy chủ phụ trợ:** đóng gói bằng **Docker** và triển khai trên nền tảng **Railway**. Máy chủ lắng nghe trên 0.0.0.0, đặt **trust proxy** để hoạt

động sau reverse proxy/HTTPS, và cung cấp endpoint `/health` cho việc giám sát trạng thái. Các tác vụ định kỳ (hết hạn đơn, tự xác nhận, xử lý no-show) chạy ngầm bằng `setInterval`.

- **Cơ sở dữ liệu: MongoDB Atlas** (dịch vụ đám mây), có sao lưu, kết nối qua chuỗi bí mật trong biến môi trường.
- **Ứng dụng di động:** biên dịch bằng **Expo (EAS Build)** thành tệp APK để cài trên thiết bị Android; trong quá trình phát triển dùng Expo Go. Ứng dụng kết nối tới máy chủ qua HTTPS.
- **Trang quản trị:** build tĩnh bằng **Vite** và phục vụ qua máy chủ web/Docker.

Cấu hình được quản lý qua biến môi trường (`.env`): cổng dịch vụ, chuỗi kết nối MongoDB, `JWT_SECRET`, danh sách CORS cho phép, và khóa API Google Distance Matrix. Tệp tải lên (ảnh đại diện, ảnh món ăn, ảnh chat) hiện được lưu cục bộ trên máy chủ và phục vụ qua đường dẫn tĩnh `/uploads`; đây là phương án “đủ dùng” cho quy mô đồ án, có thể thay bằng dịch vụ lưu trữ đối tượng (object storage) khi mở rộng.

Kết quả triển khai thử nghiệm. Hệ thống đã được triển khai và chạy ổn định ở môi trường thử nghiệm với các tài khoản mẫu cho cả bốn vai trò, kiểm chứng được trọn vẹn các luồng nghiệp vụ chính (đăng ký/OTP, đăng đơn, kết nối nhận, giao qua tình nguyện viên, xác nhận bằng mã pickup, nhắn tin và thông báo thời gian thực). Thời gian phản hồi của các API thông thường ở mức dưới 1–2 giây trong điều kiện mạng bình thường.